

Работа с отступами, со свойством float и со списками на CSS

сейчас мы с вами будем разбираться со свойствами **margin** и **padding**, которые задают *отступы* между элементами сайта. Давайте посмотрим, в чем между ними разница и как эти свойства применяются на практике.

Блок1. Свойство margin

Итак, свойство **margin** задает отступ **снаружи** от границы элемента.

Давайте посмотрим, как это выглядит на практике.

Для начала приведу вам два вложенных друг в друга блока без **margin** (они слипнутся и мы увидим двойную границу):

```
<div id="red">
  <div id="green">

  </div>
</div>
#red {
  width: 300px;
  border: 1px solid red;
}
#green {
  height: 100px;
  border: 1px solid green;
}
```

Так код будет выглядеть в браузере:

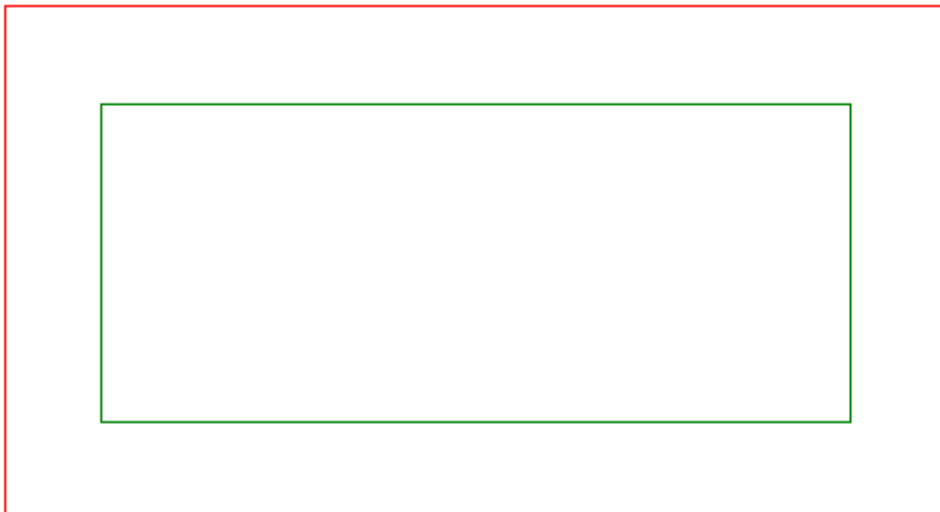


А теперь давайте зададим **margin** в 30px внутреннему блоку:

```
<div id="red">
  <div id="green">

  </div>
</div>
#red {
  width: 300px;
  border: 1px solid red;
}
#green {
  height: 100px;
  border: 1px solid green;
  margin: 30px;
}
```

Так код будет выглядеть в браузере:



Для определенной стороны

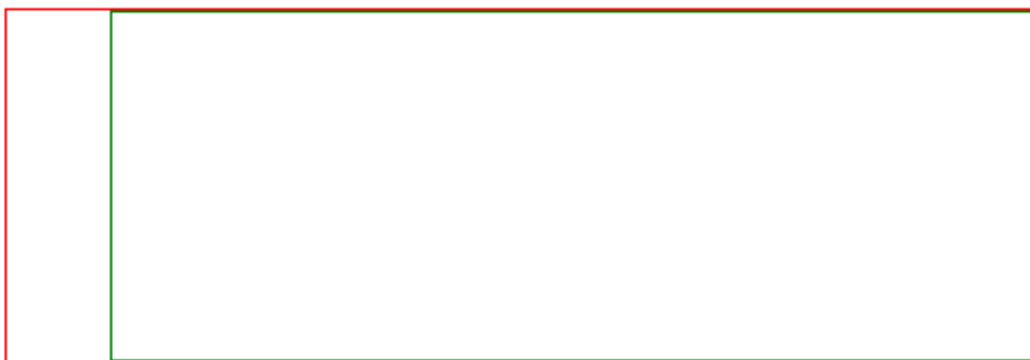
Отступ **margin** можно также задавать для определенных сторон. Для этого существуют следующие свойства: свойство **margin-left** для левого отступа, **margin-right** для правого, **margin-top** для верхнего и **margin-bottom** для нижнего.

Давайте поставим левый отступ в **30px** с помощью **margin-left**:

```
<div id="red">
  <div id="green">

  </div>
</div>
#red {
  width: 300px;
  border: 1px solid red;
}
#green {
  height: 100px;
  border: 1px solid green;
  margin-left: 30px;
}
```

Так код будет выглядеть в браузере:



Блок2. Альтернативный вариант для разных сторон

Подобно другим свойствам CSS, свойство **margin** может принимать не только одно значения, но и **два, три** или **четыре**. В этом случае разные значения задают отступы для разных сторон.

Четыре значения

Давайте посмотрим на примере, как работает **margin**, когда ему задано четыре значения. Пусть у нас сделано так: **margin: 10px 20px 30px 40px**. В этом случае 10px это отступ **сверху**, 20px - **справа**, 30px - **снизу**, 40px - **слева**. То есть мы начинаем сверху и идем по часовой стрелке: *верх-право-низ-лево*.

Этому правилу следует подавляющее большинство CSS свойств (кроме **border-radius**, у него все начинается с **верхнего левого** угла и тоже идет **по часовой стрелке**, то есть, в принципе, и тут все похоже).

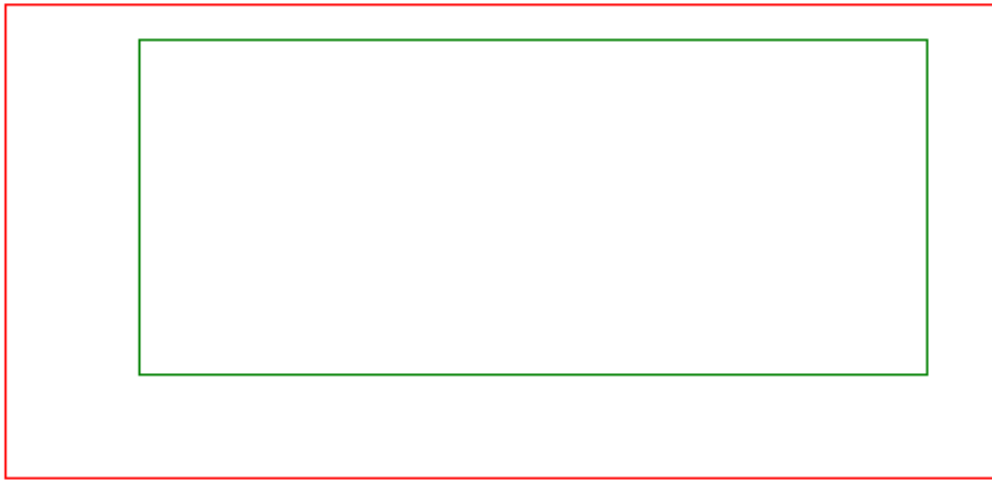
```
<div id="red">
  <div id="green">

  </div>
</div>

#red {
  width: 300px;
  border: 1px solid red;
}

#green {
  height: 100px;
  border: 1px solid green;
  margin: 10px 20px 30px 40px;
}
```

Так код будет выглядеть в браузере:



Два значения

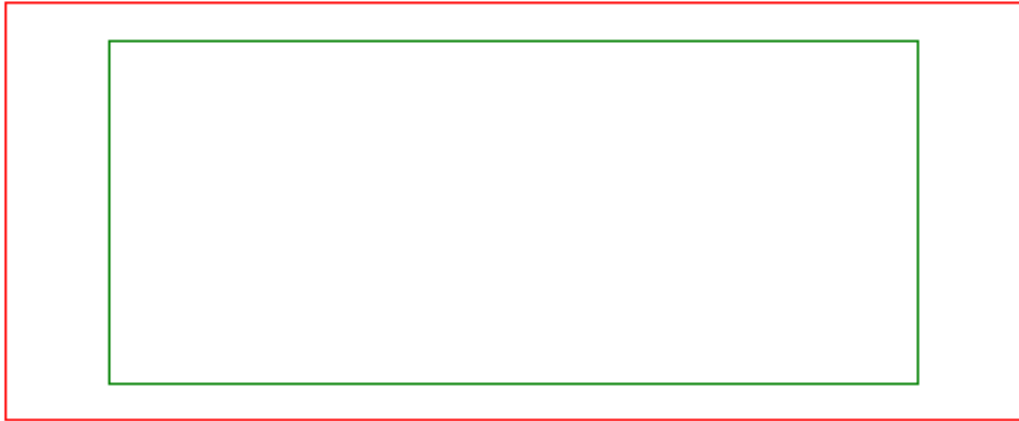
Если передать свойству **margin** два значения, то **первое** будет задавать отступ одновременно **сверху** и **снизу**, а **второе** - одновременно и **справа**, и **слева**.

Давайте посмотрим пример:

```
<div id="red">
  <div id="green">

  </div>
</div>
#red {
  width: 300px;
  border: 1px solid red;
}
#green {
  height: 100px;
  border: 1px solid green;
  margin: 10px 30px;
}
```

Так код будет выглядеть в браузере:



Значение auto

Свойство **margin** применяется не только для задания отступов, но и для **центрирования** блочных элементов. Для этого **правый** и **левый margin** следует задать в значение **auto**.

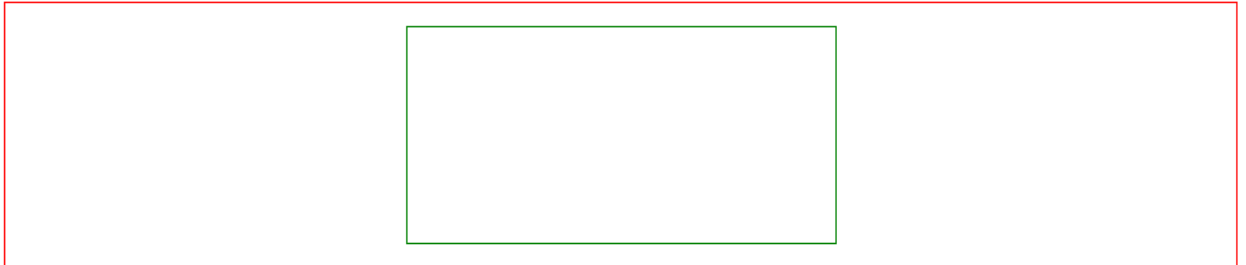
В примере ниже внутренний блок станет по центру:

```
<div id="red">
  <div id="green">

  </div>
</div>
#red {
  width: 100%;
  border: 1px solid red;
}
#green {
  height: 100px;
  width: 200px;
  border: 1px solid green;
  margin-top: 10px;
  margin-bottom: 10px;
```

```
margin-left: auto;
margin-right: auto;
}
```

Так код будет выглядеть в браузере:



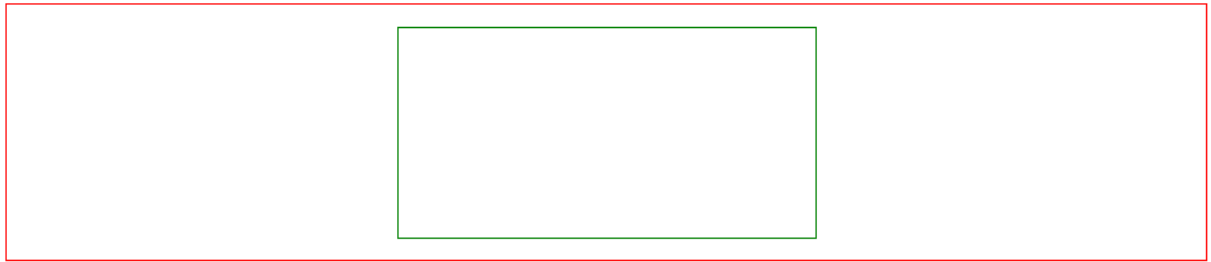
Имейте ввиду, что таким образом можно центрировать **только блочные** элементы (для строчных используйте text-align), **только по горизонтали** и только, если им **задана ширина**.

Давайте перепишем пример в сокращенной форме через одно свойство-сокращение **margin** (первое значение будет задавать верх-низ, а второе - право-лево):

```
<div id="red">
  <div id="green">

  </div>
</div>
#red {
  width: 100%;
  border: 1px solid red;
}
#green {
  height: 100px;
  width: 200px;
  border: 1px solid green;
  margin: 10px auto;
}
```

Так код будет выглядеть в браузере:

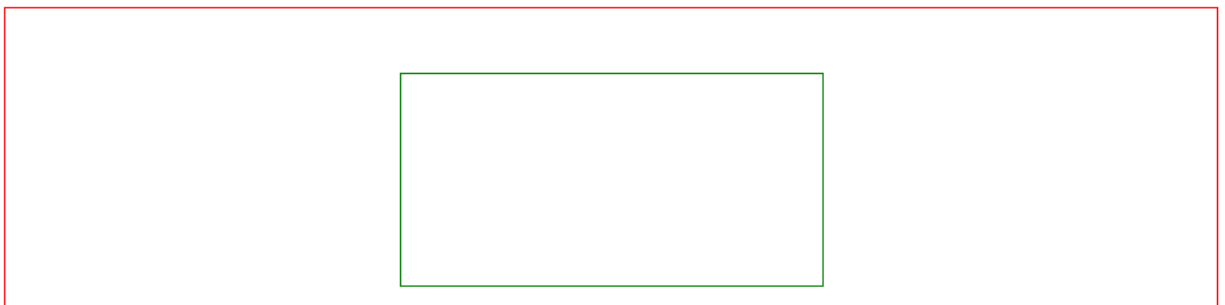


Если нам нужны разные верхний и нижний отступ **margin**, то можно передать три значения: **первое** задаст верхний отступ, **третье** - нижний, а **второе** задаст значение **auto** для правого и левого отступов:

```
<div id="red">
  <div id="green">

  </div>
</div>
#red {
  width: 100%;
  border: 1px solid red;
}
#green {
  height: 100px;
  width: 200px;
  border: 1px solid green;
  margin: 30px auto 10px;
}
```

Так код будет выглядеть в браузере:



В принципе, можно написать и четыре значения таким образом: **margin: 30px auto 10px auto**, но с **тремя** значениями писать все-таки меньше (зато сложнее понять).

Блок3. Свойство padding

Мы с вами разобрали свойство **margin**, которое делает отступ **наружу** от границы. Существует также очень похожее свойство **padding**, которое делает отступ **вовнутрь** от границы.

Работает оно по такому же принципу - есть свойства **padding-left, padding-right, padding-top** и **padding-bottom**, которые задают отступ для **разных** сторон. Также для свойства **padding** можно задавать **1, 2, 3** или **4** значения по такому же принципу, как и в **margin**.

Единственное отличие - это то, что значения **auto** для свойства **padding** не существует.

Давайте теперь посмотрим на примерах, как работает свойство **padding**. Для начала приведу блок без отступов (у него текст прилип к границам):

```
<div></div>
```

```
div {  
    width: 300px;  
    border: 1px solid red;  
    text-align: justify;  
}
```

Так код будет выглядеть в браузере:

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean a dapibus magna, ac interdum nisl. Suspendisse eget fringilla nibh, eu commodo arcu. Donec lacinia tempor velit sed tincidunt. Aliquam porttitor nulla purus, vel vulputate ipsum faucibus sed. Phasellus sodales, lorem vel cursus vehicula, ante purus lacinia dui, interdum fringilla massa eros ut dui.

А теперь зададим **padding** в **30px** для всех сторон:

```
<div></div>
```

```
div {  
    width: 300px;  
    border: 1px solid red;  
    text-align: justify;  
    padding: 30px;  
}
```

Так код будет выглядеть в браузере:

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean a dapibus magna, ac interdum nisl. Suspendisse eget fringilla nibh, eu commodo arcu. Donec lacinia tempor velit sed tincidunt. Aliquam porttitor nulla purus, vel vulputate ipsum faucibus sed. Phasellus sodales, lorem vel cursus vehicula, ante purus lacinia dui, interdum fringilla massa eros ut dui.

Свойство padding для определенной стороны

Давайте зададим отступы, используя **padding-left**, **padding-right**, **padding-top** и **padding-bottom**:

```
<div></div>
```

```
div {  
    width: 300px;  
    border: 1px solid red;
```

```
text-align: justify;
padding-top: 10px; /* сверху */
padding-right: 20px; /* справа */
padding-bottom: 30px; /* снизу */
padding-left: 40px; /* слева */
}
```

Так код будет выглядеть в браузере:

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean a dapibus magna, ac interdum nisl. Suspendisse eget fringilla nibh, eu commodo arcu. Donec lacinia tempor velit sed tincidunt. Aliquam porttitor nulla purus, vel vulputate ipsum faucibus sed. Phasellus sodales, lorem vel cursus vehicula, ante purus lacinia dui, interdum fringilla massa eros ut dui.

Блок4 Свойство padding для разных сторон

Следующие примеры показывают, как работает **padding**, если передать ему два, три или четыре параметра. В этих случаях он работает аналогично свойству **margin**, поэтому комментариев к примерам я не буду делать, просто изучите их:

Четыре значения

```
<div></div>
```

```
div {
width: 300px;
border: 1px solid red;
text-align: justify;
padding: 10px 20px 30px 40px;
```

}

Так код будет выглядеть в браузере:

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean a dapibus magna, ac interdum nisl. Suspendisse eget fringilla nibh, eu commodo arcu. Donec lacinia tempor velit sed tincidunt. Aliquam porttitor nulla purus, vel vulputate ipsum faucibus sed. Phasellus sodales, lorem vel cursus vehicula, ante purus lacinia dui, interdum fringilla massa eros ut dui.

Блок5 Расширение элемента

По умолчанию добавление **padding** расширяет элемент: если мы задали ширину **width** 100px и **padding-left** в 20px, то реальная ширина элемента будет **120px**. А если мы зададим еще и **border-left** в 10px, то реальная ширина элемента станет **130px**. То есть и padding, и border расширяют блок (и по ширине, и по высоте).

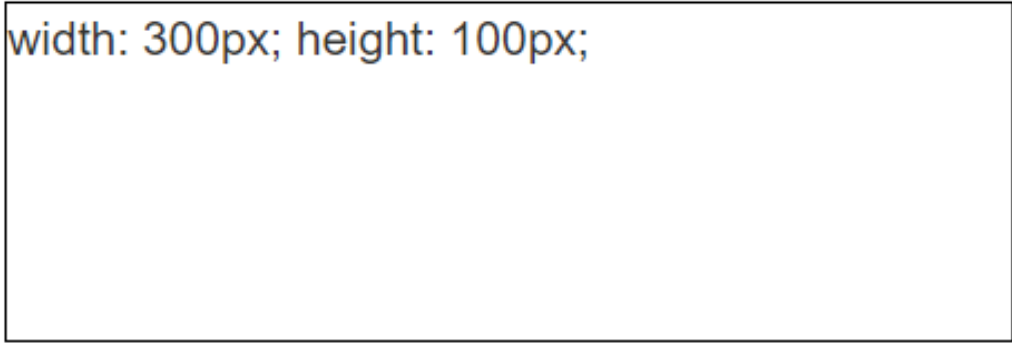
В следующем примере оба блока имеют одинаковую высоту и ширину, однако у второго блока **padding** задан, а у первого - нет. Посмотрите, как отличаются их размеры:

width: 300px; height: 100px; padding: 0px.

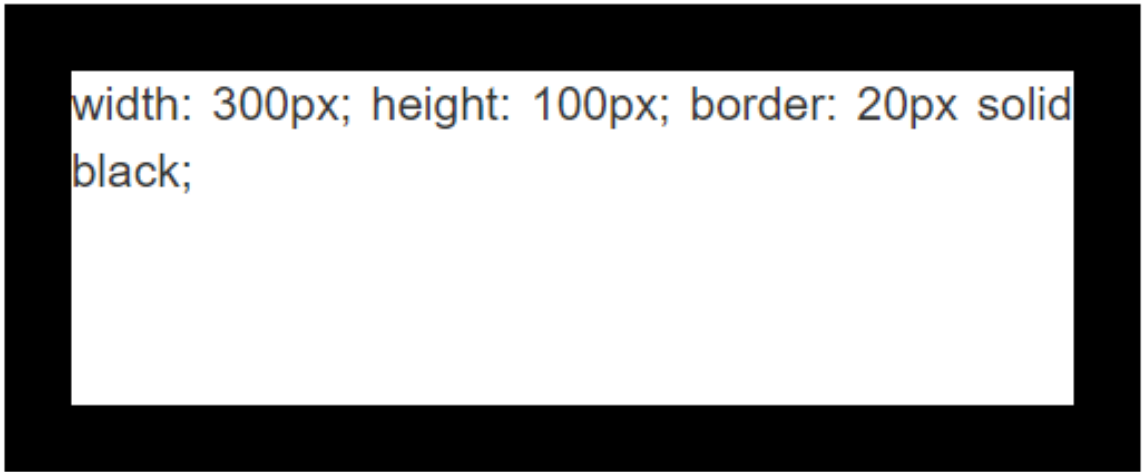
width: 300px; height: 100px; padding: 50px.

А сейчас оба блока имеют одинаковую высоту и ширину, однако у второго блока задана **граница** в 20px, а у первого - нет. Посмотрите, как отличаются их размеры:

```
width: 300px; height: 100px;
```



```
width: 300px; height: 100px; border: 20px solid  
black;
```



Блок6. Свойства **outline** и **box-sizing**

Если вам мешает расширение блоков из-за **padding** и границы, их поведение можно исправить свойствами **outline** (используется вместо границы) и **box-sizing** (отменяет расширение блоков).

Загуглите это свойство и прочитайте, оно пригодится

Блок7. Работа с маркерами списков

Мы с вами еще в начале уроков разбирались со списками **ul** и **ol** и вы наверняка помните о том, что список **ul** имеет маркеры в виде **точек**, а **ol** - в виде **цифр**. Однако, вид этих маркеров можно сменить или даже убрать их совсем.

Например, для списка **ul** можно сделать маркеры в виде **квадратов** или **кружков**, а для списка **ol** - нумерацию не только в виде цифр, а в виде **латинских** или **греческих** букв.

Делается это с помощью свойства **list-style-type**. Давайте посмотрим, какие значения оно может принимать для списка **ul**:

Значение square

Значение **square** позволяет сделать маркеры в виде **квадратиков**:

```
ul {  
    list-style-type: square;  
}  
  
<ul>  
    <li>Первый пункт.</li>  
    <li>Второй пункт.</li>  
    <li>Третий пункт.</li>  
    <li>Четвертый пункт.</li>  
    <li>Пятый пункт.</li>  
</ul>
```

Так код будет выглядеть в браузере:

- Первый пункт.
- Второй пункт.
- Третий пункт.
- Четвертый пункт.
- Пятый пункт.

Значение circle

Значение **circle** позволяет сделать маркеры в виде **кружков**:

```
ul {  
    list-style-type: circle;  
}  
  
<ul>  
    <li>Первый пункт.</li>
```

```
<li>Второй пункт.</li>
<li>Третий пункт.</li>
<li>Четвертый пункт.</li>
<li>Пятый пункт.</li>
</ul>
```

Так код будет выглядеть в браузере:

- Первый пункт.
- Второй пункт.
- Третий пункт.
- Четвертый пункт.
- Пятый пункт.

Значение **disc** делает маркеры в виде **закрашенных кружков** (это значение по умолчанию):

Значение **none** убирает маркеры совсем:

Значения для списка **ol**

Для списка **ol** дело обстоит аналогичным образом, только там применяются свои значения. Давайте, например, вместо обычных цифр сделаем **римские**:

```
ul {
    list-style-type: lower-roman;
}
<ol>
    <li>Первый пункт.</li>
    <li>Второй пункт.</li>
    <li>Третий пункт.</li>
    <li>Четвертый пункт.</li>
    <li>Пятый пункт.</li>
```


Так код будет выглядеть в браузере:

- i. Первый пункт.
- ii. Второй пункт.
- iii. Третий пункт.
- iv. Четвертый пункт.
- v. Пятый пункт.

А теперь сделаем нумерацию не числами, а маленькими латинскими буквами:

```
ul {  
    list-style-type: lower-alpha;  
}
```


- Первый пункт.
- Второй пункт.
- Третий пункт.
- Четвертый пункт.
- Пятый пункт.

Так код будет выглядеть в браузере:

- a. Первый пункт.
- b. Второй пункт.
- c. Третий пункт.
- d. Четвертый пункт.
- e. Пятый пункт.

Блок8. Маркер в виде картинки

Можно также задавать **свои маркеры**. Для этого необходимо иметь картинку, которую вы бы хотели сделать для маркеров списка, и подключить ее с помощью свойства `list-style-image: url("путь к картинке")`.

Смотрите пример:

```
ul {  
    list-style-image: url("li.gif");  
}  
<ul>  
    <li>Первый пункт.</li>  
    <li>Второй пункт.</li>  
    <li>Третий пункт.</li>  
    <li>Четвертый пункт.</li>  
    <li>Пятый пункт.</li>  
</ul>
```

Так код будет выглядеть в браузере:

- Первый пункт.
- Второй пункт.
- Третий пункт.
- Четвертый пункт.
- Пятый пункт.

Блок9. Позиция маркеров списков

По умолчанию маркеры списка не входят в сам список.

Чтобы понять, что имеется ввиду, давайте разберем следующий пример. В данном случае тегу **ul** обнулен **padding** и, казалось бы, маркеры списка должны прижиматься к границе блока, но они вылазят **наружу**! Это говорит о том, что не они реагируют на **padding**, а текст списка:

```
div {  
    border: 1px solid red;  
}  
  
ul {  
    padding: 0;  
    list-style-position: outside;  
}  
<div>  
    <ul>  
        <li>Первый пункт.</li>  
        <li>Второй пункт.</li>  
        <li>Третий пункт.</li>
```

```
        <li>Четвертый пункт.</li>
        <li>Пятый пункт.</li>
    </ul>
</div>
```

Так код будет выглядеть в браузере:

- Первый пункт.
- Второй пункт.
- Третий пункт.
- Четвертый пункт.
- Пятый пункт.

Такое поведение можно поменять, если воспользоваться свойством **list-style-position**, которое принимает два значения: **outside** (то, что мы видим по умолчанию) и значение **inside**, которое делает так, что маркеры оказываются **внутри списка**:

```
div {
    border: 1px solid red;
}

ul {
    padding: 0;
    list-style-position: inside;
}
<div>
    <ul>
        <li>Первый пункт.</li>
        <li>Второй пункт.</li>
        <li>Третий пункт.</li>
        <li>Четвертый пункт.</li>
        <li>Пятый пункт.</li>
    </ul>
```

</div>

Так код будет выглядеть в браузере:

- Первый пункт.
- Второй пункт.
- Третий пункт.
- Четвертый пункт.
- Пятый пункт.

Блок10. Свойство list-style

Существует также свойство-сокращение list-style, которое позволяет одновременно задать **тип** маркера, его **позицию** и **картинку**.

Блок11. Ограничение высоты и ширины

Свойство min-width устанавливает минимальную ширину элемента. Значением свойства служат любые **единицы для размеров** либо ключевое слово none, означающее отсутствие значения.

Обычно задается для элемента с плавающей шириной (ширина задана в процентах или не задана вообще, а элемент по ширине раздвигается своим содержимым). Если ширина элемента задана в процентах - при уменьшении окна браузера она будет уменьшаться, пока не достигнет значения min-width.

Синтаксис

```
селектор {  
    max-width: значение;  
}
```

Пример1

В данном примере ширина составляет процент от ширины родителя. Уменьшите окно браузера по ширине, и контейнер тоже уменьшится, чтобы подстроиться под ширину окна браузера. Однако он будет уменьшаться только до величины, определенной в min-width, это 400px. Как только контейнер достигнет этой ширины - он перестанет уменьшаться:

```
<div id="elem"></div>#elem {  
    width: 70%;  
    min-width: 400px;
```

```
height: 300px;
border: 1px solid black;
}
```

Пример2

В данном примере ширина контейнера ограничена с обеих сторон: он не сможет стать больше 900px и меньше 400px:

```
<div id="elem"></div>#elem {
width: 70%;
min-width: 400px;
max-width: 900px;
height: 100px;
border: 1px solid black;
}
```

Свойство max-width устанавливает максимальную ширину элемента. Значением свойства служат любые **единицы для размеров** либо ключевое слово none, означающее отсутствие значения.

Обычно свойство задается для элемента с плавающей шириной (ширина задана в процентах или не задана вообще, а элемент по ширине раздвигается своим содержимым). Если ширина элемента задана в процентах - при увеличении окна браузера она будет увеличиваться, пока не достигнет значения max-width.

Синтаксис

```
селектор {
max-width: значение;
}
```

Пример1

В данном примере ширина составляет процент от ширины родителя. Увеличьте окно браузера по ширине, и контейнер тоже увеличиться, чтобы подстроиться под ширину окна браузера. Однако он будет увеличиваться только до величины, определенной в max-width, это 900px. Как только контейнер достигнет этой ширины - он перестанет увеличиваться:

```
<div id="elem"></div>#elem {
width: 70%;
max-width: 900px;
height: 300px;
border: 1px solid black;
}
```

Пример2

В данном примере ширина контейнера ограничена с обеих сторон: он не сможет стать больше 900px и меньше 400px:

```
<div id="elem"></div>#elem {  
    width: 70%;  
    max-width: 900px;  
    min-width: 400px;  
    height: 300px;  
    border: 1px solid black;  
}
```

Свойство min-height устанавливает минимальную высоту элемента. Значением свойства служат любые единицы для размеров либо ключевое слово none, означающее отсутствие значения.

Обычно задается для элемента с плавающей шириной (высота задана в процентах или не задана вообще, а элемент по высоте раздвигается своим содержимым).

Если задана минимальная высота - то меньше этой высоты элемент не сможет стать. Если текста меньше, чем нужно для min-height, элемент будет иметь высоту min-height.

Если же текста больше - высота элемента вычислится в зависимости от количества текста.

Синтаксис

```
селектор {  
    min-height: значение;  
}
```

Пример1

В данном примере высота не установлена вообще и будет рассчитана браузером самостоятельно в зависимости от количества текста. Однако, так как установлено свойство min-height, то высота меньше этого значения не уменьшится, даже если текста в нем совсем не будет:

```
<div id="elem">  
    Lorem ipsum dolor sit amet.  
</div>#elem {  
    min-height: 100px;  
    width: 300px;  
    border: 1px solid black;
```

```
}
```

Свойство `max-height` устанавливает максимальную высоту элемента. Значением свойства служат любые [единицы для размеров](#) либо ключевое слово `none`, означающее отсутствие значения.

Обычно задается для элемента с плавающей шириной (высота задана в процентах или не задана вообще, а элемент по высоте раздвигается своим содержимым). Если задана максимальная высота - то больше этой высоты элемент не сможет стать.

Синтаксис

```
селектор {  
    max-height: значение;  
}
```

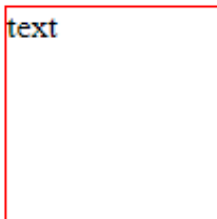
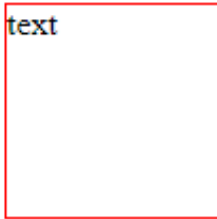
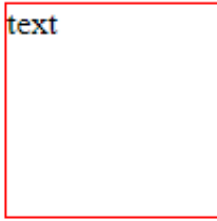
Блок12. Свойство `display`

Вы уже знаете, что все теги относятся к определенной блочной модели. При желании, однако, можно сменить модель с помощью свойства `display`. С помощью значения `block` данного свойства элемент можно сделать блочным, а с помощью значения `inline` - строчным.

Давайте сделаем так, чтобы теги `span` вели себя как блочные элементы:

```
<span>text</span>  
<span>text</span>  
<span>text</span>span {  
    display: block;  
    width: 100px;  
    height: 100px;  
    border: 1px solid red;  
    margin-bottom: 20px;  
}
```

Результат выполнения кода:



А теперь сделаем так, чтобы теги `div` вели себя как строчные:

```
<div>text</div>
<div>text</div>
<div>text</div>div {
    display: inline;
    width: 100px;
    height: 100px;
    border: 1px solid red;
}
```

Результат выполнения кода:

text text text

Блок13. Свойство float

Пусть у нас есть див с каким-то длинным текстом. Давайте вставим в начало этого текста картинку:

```
<div>
  
  длинный текст
</div>div {
  width: 400px;
  text-align: justify;
}
```

Результат выполнения кода:



Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean a dapibus magna, ac interdum nisl. Suspendisse eget fringilla nibh, eu commodo arcu. Donec lacinia tempor velite sed tincidunt. Aliquam porttitor nulla purus, vel vulputate ipsum faucibus sed. Phasellus sodales, lorem vel cursus vehi ante purus lacini dui, interdum fringilla massa eros ut dui. Nullam et ipsum ex. Donec ut iaculis massa. Duis vel ullamcorper per dolor. Suspendisse tristique dolor risus, nec laoreet du aliquet maximus. Maecenas tempus congue arcu, si amet auctor dui efficitur in. Proin ac auctor nisi. Phasellus luctus eur neque sollicitudin rutrum. Curabitur ipsum justo, sodales in lectus non, molestie ullamcorper elit, orciro varius natoque.

Как вы видите, низ картинки располагается на первой строке текста, а остальная часть уходит вверх. Справа от картинки получается большое пустое пространство. Давайте сделаем так, чтобы текст заполнил это пустое пространство. Для этого картинке поставим свойство float в значении left:

```
<div>
  
  длинный текст
</div>div {
  width: 400px;
  text-align: justify;
}
img {
  float: left;
}
```

Результат выполнения кода:

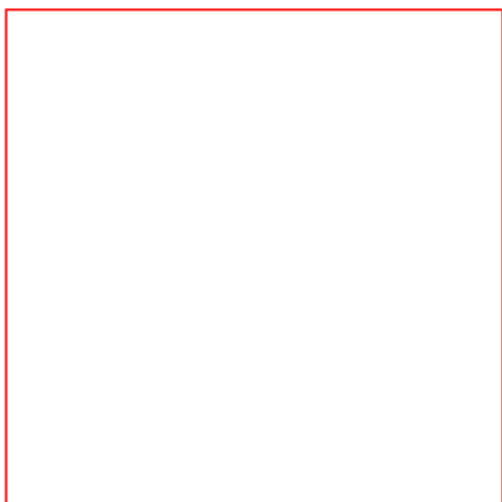


Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean a dapibus magna, ac interdum nisl. Suspendisse eget fringilla nibh, eu commodo arcu. Donec lacinia tempor velite sed tincidunt. Aliquam porttitor nulla purus, vel vulputate ipsum faucibus sed. Phasellus sodales, lorem vel cursus vehi ante purus lacini dui, interdum fringilla massa eros ut dui. Nullam et ipsum ex. Donec ut iaculis massa. Duis vel ullamcorper per dolor. Suspendisse tristique dolor risus, nec laoreet du aliquet maximus. Maecenas tempus congue arcu, si amet auctor dui efficitur in. Proin ac auctor nisi. Phasellus luctus eur neque sollicitudin rutrum. Curabitur ipsum justo, sodales in lectus non, molestie ullamcorper elit, orciro varius natoque.

Практическая работа 6

Отступы, списки, флоты

1 Повторите страницу по данному образцу:



Lorem ipsum dolor sit amet,
consectetur adipiscing elit.
Nulla pharetra risus erat, eu
imperdiet felis suscipit at.
Vivamus rhoncus vel neque
non eleifend. Donec adipiscing
eros quis molestie sodales.
Proin vitae nulla ut sem
lobortis scelerisque. Aliquam
ultrices porta sem quis
fringilla. Quisque pretium
enim ut elit fringilla, in
adipiscing ligula placerat.
Integer id tortor tristique,
facilisis lorem sit amet, porta
ante. Nulla eget diam in erat
hendrerit gravida.

2 Повторите макет сайта по образцу:

Шаблон сайта

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nulla pharetra risus erat, eu imperdiet felis suscipit at. Vivamus rhoncus vel neque non eleifend. Donec adipiscing eros quis molestie sodales. Proin vitae nulla ut sem lobortis scelerisque. Aliquam ultricies porta sem quis fringilla. Quisque pretium enim ut elit fringilla, in adipiscing ligula placerat. Integer id tortor tristique, facilisis lorem sit amet, porta ante. Nulla eget diam in erat hendrerit gravida.

Cras ut nunc metus. Nullam id faucibus ante. Proin tristique lorem tristique dolor porttitor, eget adipiscing leo gravida. Proin erat erat, laoreet sit amet tincidunt ac, iaculis nec libero. In placerat odio ac dapibus faucibus. Praesent ut est in ligula facilisis congue eget ac ante. Phasellus at felis vitae est molestie ultricies.

Integer odio risus, imperdiet vitae orci non, pulvinar suscipit dui. Proin condimentum luctus nulla, ut bibendum mi convallis nec. Vestibulum at venenatis nulla. Vivamus ligula justo, commodo eget quam a, aliquam semper dolor. Donec dapibus nunc hendrerit turpis porta auctor. Maecenas in hendrerit felis. Quisque id ligula a dui rhoncus sodales ac eu nulla. Vestibulum ac dignissim nulla. Fusce sollicitudin nisl massa, quis viverra metus condimentum non. Fusce quis viverra enim, id viverra libero. In accumsan justo enim, fringilla egestas ipsum suscipit at. Etiam sed quam mi. Vestibulum ultricies nisi luctus convallis malesuada.

3.Повторите резиновый макет по образцу(подсказки дам)

Резиновый шаблон сайта

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nulla pharetra risus erat, eu imperdiet felis suscipit at. Vivamus rhoncus vel neque non eleifend. Donec adipiscing eros quis molestie sodales. Proin vitae nulla ut sem lobortis scelerisque. Aliquam ultricies porta sem quis fringilla. Quisque pretium enim ut elit fringilla, in adipiscing ligula placerat. Integer id tortor tristique, facilisis lorem sit amet, porta ante. Nulla eget diam in erat hendrerit gravida.

Cras ut nunc metus. Nullam id faucibus ante. Proin tristique lorem tristique dolor porttitor, eget adipiscing leo gravida. Proin erat erat, laoreet sit amet tincidunt ac, iaculis nec libero. In placerat odio ac dapibus faucibus. Praesent ut est in ligula facilisis congue eget ac ante. Phasellus at felis vitae est molestie ultricies.

Integer odio risus, imperdiet vitae orci non, pulvinar suscipit dui. Proin condimentum luctus nulla, ut bibendum mi convallis nec. Vestibulum at venenatis nulla. Vivamus ligula justo, commodo eget quam a, aliquam semper dolor. Donec dapibus nunc hendrerit turpis porta auctor. Maecenas in hendrerit felis. Quisque id ligula a dui rhoncus sodales ac eu nulla. Vestibulum ac dignissim nulla. Fusce sollicitudin nisl massa, quis viverra metus condimentum non. Fusce quis viverra enim, id viverra libero. In accumsan justo enim, fringilla egestas ipsum suscipit at. Etiam sed quam mi. Vestibulum ultricies nisi luctus convallis malesuada.

У вас будет 1 div элемент в котором у вас h1 и параграфы(p).

Для этого дива используйте свойства(width: **ШИРИНА в процентах**
min-width:**ВАШИ РАЗМЕРЫ**
max-width: **ВАШИ РАЗМЕРЫ**

```
margin: 50px auto;  
padding: 30px;  
text-align: justify;  
border: ВАШИ РАЗМЕРЫ  
border-radius: ВАШИ РАЗМЕРЫ
```

После выполнения задания поиграйтесь с окном браузера(уменьшайте и увеличивайте ширину просмотра и сравните ваш прошлый макет с этим)

4 повторить страницу, используйте float для картинок.текст произвольный



Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nulla pharetra risus erat, eu imperdiet felis suscipit at. Vivamus rhoncus vel neque non eleifend. Donec adipiscing eros quis molestie sodales. Proin vitae nulla ut sem lobortis scelerisque. Aliquam ultricies porta sem quis fringilla. Quisque pretium enim ut elit fringilla, in adipiscing ligula placerat. Integer id tortor tristique,

facilisis lorem sit amet, porta ante. Nulla eget diam in erat hendrerit gravida. Cras ut nunc metus. Nullam id faucibus ante. Proin tristique lorem tristique dolor porttitor, eget adipiscing leo gravida. Proin erat erat, laoreet sit amet tincidunt ac, iaculis nec libero. In placerat odio ac dapibus faucibus. Praesent ut est in ligula facilisis congue eget ac ante. Phasellus at felis vitae est molestie ultricies. Integer odio risus, imperdiet vitae orci non, pulvinar suscipit dui. Proin condimentum luctus nulla, ut bibendum mi convallis nec. Vestibulum at venenatis nulla. Vivamus ligula justo, commodo eget quam a, aliquam semper dolor. Donec dapibus nunc hendrerit turpis porta auctor. Maecenas in hendrerit felis. Quisque id ligula a dui rhoncus sodales ac eu nulla. Vestibulum ac dignissim nulla. Fusce sollicitudin nisl massa, quis viverra metus condimentum non. Fusce quis viverra enim, id viverra libero. In accumsan justo enim, fringilla egestas ipsum suscipit at. Etiam sed quam mi. Vestibulum ultricies nisi luctus convallis malesuada.

